

Compliance Notification Automation — Project Brief

Prepared for the meeting with Vincent (Business Intelligence, Mobile/Bi Specialist)

The one-sentence summary

A Python tool reads our security compliance reports (CrowdStrike, Purview/DLP, Zscaler), figures out which colleague owns each non-compliant device, and sends them an automated reminder email. It's fully built and tested against real data. What's missing is a proper place for it to run in Azure, and permission to actually send mail.

1. The problem this solves

Several security tools (CrowdStrike, Microsoft Purview, Zscaler) each produce their own list of non-compliant devices. Today, someone manually cross-references these lists, figures out who owns each machine, and chases them individually. The goal is to automate that: read the reports, work out the owner, send a plain-language reminder.

2. What was actually hard about this

Not the emailing — the **data**. Five different report files use different column names for the same things, some devices appear in more than one report (and must not be double-counted or double-emailed), and matching a device's owner (a free-text name from our asset system) to a real email address (in our AD directory) turned out to be the hardest part of the whole project, because names are written inconsistently — "Smith, John" vs "John Smith" vs "J. Smith-JS" — and some of those are genuinely ambiguous (two different people can share a name).

3. How the pipeline works

See the pipeline diagram shown in chat. In short: five report files get read, cleaned, and merged into one de-duplicated list of non-compliant devices. Each device's owner name is matched against the AD directory. A composed message is generated per person (not per device — someone with three non-compliant machines gets one email, not three). The message is deliberately simple: it does not name the technical issue (agent outdated, DLP not enrolled, etc.) — it just tells the user to power their device on and stay connected for a few hours so pending updates can install. The full technical detail is kept in an internal spreadsheet for whoever handles remediation, not sent to the user.

4. The most important design decision: never guess

See the recipient-resolution diagram shown in chat. When a device's owner name can't be matched to a directory entry with confidence, the tool **never sends an email based on a guess**. It has three outcomes:

- **Confident match** → email sent automatically.

- **Ambiguous match** (e.g. two people with the same name) → held in a "Review" list for a human to resolve once, which then gets remembered permanently via a manual override file.
- **No match at all** → logged for visibility, nobody emailed.

This is enforced by an automated test that specifically checks an ambiguous name can never end up in the "send" list — it's the single most important safety property in the whole system.

5. Safety guardrails around actually sending

Nothing has ever been sent to a real colleague. The sending tool has three explicit modes:

- `--dry-run` (default) — shows what would be sent, sends nothing.
- `--send-to-self` — really sends via Microsoft Graph, but every message is redirected to a test inbox, with a banner showing who it would really have gone to. This is how the send mechanism gets verified before anyone else sees anything.
- `--send-live` — sends to real people. Requires an extra, explicit confirmation flag to run at all.

On top of that: a cap on how many recipients can be emailed in one run (to catch a misconfiguration before it reaches everyone), an audit log of every send attempt, and a check that stops the same person being emailed twice in one day.

6. Testing discipline

The codebase has 100+ automated tests, each one built to reproduce a specific real bug encountered during development (not hypothetical cases). Every fix was verified by deliberately reintroducing the bug, confirming the test caught it, then reverting — so the tests are proven to actually work, not just present. A pre-commit hook now runs the full suite automatically before every code change is saved, so a regression can't be accidentally shipped.

7. Current status

Built, tested, working locally:

- Full data pipeline (reconciliation, dedup, compliance rules)
- Recipient resolution (name matching, review/override workflow)
- Message composition (plain text and HTML)
- Send mechanism with full guardrails (currently untestable end-to-end — see below)
- 100+ automated tests, pre-commit hook

Blocked, waiting on others:

- **Microsoft Graph `Mail.Send` permission** — an Azure AD app registration needs this permission granted with admin consent before any real email can be sent. Requested

from Kamal; status not yet confirmed.

- **A proper place to run this** — this is what today's meeting is about. It currently runs on a personal laptop for development/testing only; Kamal was clear that production must run in an AIA-controlled environment.

8. The proposed target architecture (Vincent's design)

See the architecture diagram shown in chat. In plain terms:

Component	Role
Storage account (Blob)	Where the report files get uploaded — replaces the local folder used today
Function app	Where the actual code runs, on a schedule, instead of on a laptop
Managed identity	The Function App's own identity — lets it be granted access to Storage and Key Vault without any password being stored
Key Vault	Securely holds the Graph credential — the code fetches it at runtime, never hardcoded
Service principal	The identity used to actually call Microsoft Graph and send mail

This matches the "Azure Function" option we were already expecting to need — Vincent has filled in the specific storage and secrets pattern for it.

9. Key terms glossary

- **Function App** — where code runs in Azure, triggered on a schedule instead of run manually.
- **Storage Account / Blob Storage** — a cloud file store; replaces a local folder.
- **Key Vault** — a secure store for secrets/passwords; replaces a local `.env` file.
- **Managed Identity** — an identity Azure gives to a piece of infrastructure (like the Function App itself), so it can be granted permissions without a password ever being typed or stored.
- **App Registration / Service Principal** — the identity the code authenticates as when it calls Microsoft Graph.
- **Microsoft Graph** — Microsoft's API for interacting with Outlook/Teams/365 data programmatically.
- **Mail.Send (Application permission)** — the specific permission that lets an app send email with no human signed in; requires admin consent to grant.

10. Questions to ask in the meeting

- What's the current status of the `Mail.Send` app registration and admin consent request with Kamal?
- How do I get access to the Function App and Storage Account — portal access, or does the team deploy on my behalf?
- Is there an existing template/example Function App at AIA to model this on?
- What does day-to-day development look like once this is set up — fast iteration, or a slower deploy cycle?
- Any cost or approval process to be aware of for provisioning these resources?
- Does the "AIA-controlled environment" requirement apply to development and testing too, or only to live sending?

11. What NOT to commit to in this meeting

- A go-live date — the timeline depends on access/provisioning that hasn't happened yet.
- Any opinion on the architecture itself — Vincent is the specialist here; the goal is to understand and align, not critique.
- Scope additions (e.g. Teams notifications) — deliberately deferred to a later phase once email is live and stable, so as not to complicate the current, already-multi-step approval process.